



**SWAMINARAYAN
UNIVERSITY**
THE RESERVOIR OF ALL KNOWLEDGE

**Faculty of Engineering (Degree)
Semester – 7th**

Subject Name – Information security

Subject Code – BE103707

Faculty Name – Radhika Gokani

CHAPTER -7

1. Digital Signature, its properties, requirements and security

- A **Digital Signature** is a **cryptographic technique** used to ensure the **authenticity, integrity, and non-repudiation** of a digital message or document.
- It acts like an electronic form of a handwritten signature but is **much more secure**.
- A **Digital Signature** is a cryptographic value generated using the sender's **private key** that verifies:
 - **Message authenticity** (sender is genuine),
 - **Data integrity** (message not altered),
 - **Non-repudiation** (sender cannot deny sending it).

How a Digital Signature Works

1. Signature Creation

1. Sender creates a **hash** of the message using a hashing algorithm (e.g., SHA-256).
2. The hash is encrypted using the **sender's private key**.
3. The encrypted hash = **Digital Signature**.
4. The signature is attached to the message and sent.

2. Signature Verification

1. Receiver decrypts the signature using the **sender's public key**.
2. Receiver generates a fresh hash of the received message.
3. If both hashes match →
 - ✓ message is authentic
 - ✓ no tampering occurred

Applications of Digital Signatures

- Secure email (e.g., S/MIME)
- Software distribution and updates
- Online banking and financial transactions
- e-Governance (Aadhaar e-sign, e-filing)
- Blockchain and cryptocurrencies
- Legal digital documents (contracts, certificates)

Advantages

- Enhances security
- Prevents forgery
- Ensures data integrity
- Legally valid in many countries

Difference: Digital Signature vs. Electronic Signature

Digital Signature	Electronic Signature
Cryptographic, secure	Simple electronic form of signature
Ensures integrity & authentication	Only indicates approval
Legally strong	May not always be legally binding

Properties of a Digital Signature

1. Authenticity

- Verifies the identity of the sender.
- Ensures the message is from a legitimate source.

2. Integrity

- Ensures the message has not been altered during transmission.
- Any change in the content changes the hash and invalidates the signature.

3. Non-Repudiation

- Sender cannot deny sending or signing the message.
- Signature is uniquely tied to the sender's private key.

4. Verifiability

- Receiver can independently verify the signature using the sender's **public key**.

5. Unforgeability

- Without the sender's private key, no one can generate a valid signature.

6. Efficiency

- Verification is fast and does not require decrypting the entire message (only the hash).

Requirements for a Secure Digital Signature

For a digital signature to be effective and trustworthy, it must satisfy the following requirements:

1. Depends on Public-Key Cryptography

- Uses a pair of keys:
 - **Private key** → for signing
 - **Public key** → for verification

2. Uses Hash Functions

- A secure hash (SHA-256, SHA-512) must be used.
- Hash function should be **collision-resistant** (no two different messages produce the same hash).

3. Signature Must Be Unique

- Even if the message is slightly modified, the signature must change.

4. Infeasible to Forge

- No attacker should be able to create a valid signature without knowing the private key.

5. Binding to User

- Signature must be mathematically linked to the signer's identity.

6. Verifiable by Any Third Party

- Anyone with the public key should be able to validate the signature.

7. Should Not Reveal the Private Key

- Performing verification should not compromise the signer's private key.

Security of Digital Signatures

Digital signatures provide strong security through **cryptographic algorithms**, **hash functions**, and **key management**. Security depends on the following components:

A. Cryptographic Security

1. Private Key Security

- The entire system depends on keeping the private key secret.
- If the private key is exposed, signatures can be forged.

2. Strong Algorithms

Common secure signature algorithms:

- **RSA**
- **DSA**
- **ECDSA**
- **EdDSA**

These algorithms are designed to resist brute-force and mathematical attacks.

B. Hash Function Security

Secure hash functions ensure:

- **Collision resistance:** No two messages have the same hash.
- **Preimage resistance:** Cannot find a message from its hash.
- **Second preimage resistance:** Cannot find another message with the same hash.

If hashing is weak → signatures can be forged.

C. Certificate Authorities (CA) & PKI

Digital signatures used in real world often depend on:

- **Digital Certificates**
- **Certificate Authorities (CA)**
- **Public Key Infrastructure (PKI)**

These verify the identity of the signer and prevent impersonation attacks.

D. Protection Against Attacks

Digital signatures protect against:

- **Man-in-the-Middle attacks**
- **Forgery of identity**
- **Message tampering**
- **Replay attacks**
- **Impersonation**

Because the signature is mathematically tied to both:

- the **private key**, and
- the **exact message content**

any mismatch makes the signature invalid.

2. various digital signature schemes (ElGamal and Schnorr)

Digital Signature Schemes are cryptographic algorithms that allow a user to sign digital data using a **private key**, and others can verify the signature using the **public key**.

Two widely used signature schemes based on **Discrete Logarithm Problem (DLP)** are:

1. **ElGamal Digital Signature Scheme**
2. **Schnorr Digital Signature Scheme**

1. ElGamal Digital Signature Scheme

ElGamal signature is based on the difficulty of solving the **Discrete Logarithm Problem** in a finite field.

It is similar to the ElGamal encryption algorithm but used for signing.

Steps of ElGamal Signature Scheme

A. Key Generation

1. Choose a large prime number **p**.
2. Select a generator **g** of the multiplicative group modulo **p**.
3. Choose a private key **x** such that:
 $1 < x < p - 1$
4. Compute public key:
 $y = g^x \bmod p$

Public key: (p, g, y)

Private key: x

B. Signing Process

To sign a message **M**:

1. Choose a random **k** such that $\gcd(k, p-1) = 1$
2. Compute
 $r = g^k \bmod p$
3. Compute message hash: **H(M)**
4. Compute
 $s = (H(M) - x \cdot r) \cdot k^{-1} \bmod (p-1)$

Signature = (r, s)

C. Verification

To verify signature (r, s):

Check if:

$$y^r \cdot r^s \bmod p = g^{H(M)} \bmod p$$

If both sides are equal $\rightarrow$ signature is valid.

Features of ElGamal Scheme

- Based on discrete logarithm.
- Signature has **two components** (r, s).
- Requires **random k**, must not be reused.
- Larger signature size compared to Schnorr.

2. Schnorr Digital Signature Scheme

Schnorr signature is also based on the **Discrete Logarithm Problem**, but is more **efficient**, **compact**, and widely used in modern systems such as **Bitcoin (Taproot)**.

It is known for:

- Small signatures
- Fast verification
- Strong security under DLP

Steps of Schnorr Signature Scheme

A. Key Generation

1. Choose large prime **p** and subgroup prime **q** where $q \mid (p-1)$.
2. Choose generator **g** of subgroup of order **q**.
3. Select private key **x**, where:
 $0 < x < q$

4. Compute public key:

$$y = g^x \bmod p$$

Public key: (p, q, g, y)

Private key: x

B. Signing Process

To sign message **M**:

1. Choose random **k** ($0 < k < q$)
2. Compute commitment:
 $r = g^k \bmod p$
3. Compute challenge hash:
 $e = H(M \parallel r)$
4. Compute response:
 $s = (k + x \cdot e) \bmod q$

Signature = (e, s)

C. Verification

To verify:

1. Compute:
 $r' = g^s * y^{-e} \bmod p$
2. Compute:
 $e' = H(M \parallel r')$
3. If $e == e'$, signature is valid.

Features of Schnorr Scheme

- Very **small signature** (just two numbers: e and s).
- More efficient than ElGamal.
- Strong security due to collision-resistant hashing.
- Resistant to many known attacks.
- Widely used in blockchain and modern cryptography.

Comparison: ElGamal vs. Schnorr Signature

Feature	ElGamal	Schnorr
Mathematical base	Discrete Logarithm	Discrete Logarithm
Signature size	Larger (r, s)	Smaller (e, s)
Efficiency	Moderate	Very efficient
Security	Good, but depends on random k	Strong, tighter proofs
Use cases	Early digital signature systems	Bitcoin, modern cryptosystems
Hashing	Used in calculating s	Used to compute challenge e

3. NIST digital Signature algorithm

The **NIST Digital Signature Algorithm (DSA)** is a **Federal Information Processing Standard (FIPS 186)** developed by the **National Institute of Standards and Technology (NIST)**.

It is a widely used **public-key digital signature scheme** based on the **Discrete Logarithm Problem (DLP)**.

DSA is used to **generate and verify digital signatures** to ensure:

- **Authentication**
- **Integrity**
- **Non-repudiation**

It is not used for encryption — only for signing.

1. Concepts

DSA uses:

- **Public key** → for verification
- **Private key** → for signing
- **Hash function** → SHA-256, SHA-384, SHA-512 (in updated standards)

Security relies on the hardness of computing **discrete logarithms** modulo a large prime.

2. Parameters Used in DSA

Global Public Parameters:

1. **p** – Large prime (1024 bits, 2048 bits, or 3072 bits)
2. **q** – Prime divisor of $(p - 1)$ (160/224/256 bits)
3. **g** – Generator of order q modulo p

These are shared by all users in a system.

3. Key Generation

Step 1: Choose private key

- Select **x**, where
 $0 < x < q$

Step 2: Compute public key

- Compute
 $y = g^x \bmod p$

◆ Private key: **x**

◆ Public key: **(p, q, g, y)**

4. Signature Generation (Signing)

To sign a message **M**:

Step 1: Compute hash

- Compute message digest
 $H = \text{Hash}(M)$
(e.g., SHA-256)

Step 2: Choose random k

- Select random per-message key **k** with
 $0 < k < q$
- NOTE: **k must never repeat**, or security breaks.

Step 3: Compute:

- $r = (g^k \bmod p) \bmod q$
- $s = (k^{-1} \times (H + x \cdot r)) \bmod q$

Signature = (r, s)

5. Signature Verification

Given signature **(r, s)** and message **M**:

Step 1: Compute hash

$H = \text{Hash}(M)$

Step 2: Compute:

- $w = s^{-1} \bmod q$
- $u1 = (H \times w) \bmod q$
- $u2 = (r \times w) \bmod q$

Step 3: Compute verifier value

- $v = [(g^{u1} \times y^{u2}) \bmod p] \bmod q$

If $v = r \rightarrow$ Signature is VALID

6. Security of DSA

Security Based On

- Difficulty of **Discrete Logarithm Problem (DLP)**
- Hash function strength (SHA-2 family)
- Random per-message number **k**
(If k repeats $\rightarrow$ private key can be recovered)

Security Strengths

- Provides **authentication, integrity, non-repudiation**
- Supported in many standards (TLS, SSH, PGP)
- Efficient signature generation

Security Weaknesses

- Requires a **strong random k**
- Verification is slower compared to RSA
- Key sizes must be large to remain secure

7. Advantages of DSA

- Official NIST standard (FIPS 186)
- Efficient for signing (fast exponentiation)
- Smaller signature sizes (compared to ElGamal)
- Open and widely supported

8. Applications

- Digital certificates (X.509)
 - Secure email (S/MIME)
 - Secure Shell (SSH)
 - Government and military systems
 - Software signing
-